

Numerical Analysis (089180)
Prof. Carlo de Falco
carlo.defalco@polimi.it

A.Y. 2024-2025
Giuseppe Calìo
giuseppe.calio@polimi.it

Lab 1 – Solutions

October 9, 2024

1 Introduction

- info about me:
 - PhD student in Mathematical Models and Methods (Mathematics Department)
 - Research topics: Advanced Modeling of Ionic Propulsion for Atmospheric Flight
 - email: `giuseppe.calio@polimi.it`
- 10 labs = 20 h
- material on `webeep.polimi.it`: lecture slides and printable A4 version(.pdf), lecture exercises (.m), homework exercises(.m);
- approximation of function zeros, functions and integrals, numerical resolution of linear systems and ODEs

2 The lab tool: MATLAB

We want to focus on the mathematics with low programming effort. **MATLAB**(MATrix LABoratory) is an interactive software for:

- scientific computing
- statistical analysis
- vector and matrix computations
- symbolic computation
- graphics

Some **MATLAB** characteristics:

- graphical user interface;
- no need of compiling;
- code modularity: functions and scripts;
- simple graphics commands;
- I/O: loading and saving files, variables, and data.

Open source alternative: **Octave** - www.octave.org

2.1 GUI description

- **Command window:** where you digit the commands and visualize the results;
- **Command history:** there you can visualize previously used commands and re-execute them in the command line by just clicking on them;
- **Current folder:** MATLAB has a working directory. It looks in it for the functions that are not native and it saves in it by default. You can edit functions from there. You can change it also from the upper bar;
- **Workspace:** it displays all the created variables with their type and size;
- **Editor:** here you can write a script or a function and save it as an .m file in your working directory. Then you can execute/call it from the command line as usual.

2.2 Basic MATLAB usage

```
% a) Assignment operation:
a = 1 % with echo
b = 2; % without echo

% b) Recall previous commands:
% You can recall previously used commands with up and down arrows. If you already know a part
of the command you want to recall, you can digit it and then use up and down arrows.

% c) Scripts:
% When using the editor you can write the script and then execute it from the command line.
Remember, the script has to be in the working directory in order to be executed.

% d) Save/load history or the current status of the workspace
% Current workspace variables
whos % will display all the variables
whos a % will display only the variable a
% Save
diary matlab_intro.txt % saves the history
save matlab_intro % saves the whole workspace
a = 3;
save matlab_intro_ONLYa a % saves only the variables a, but no the variable b
% Clear the workspace
clear a % clears only the variable a
whos a % a does not exists anymore
clear all % clears every remaining variable (in this case, the variable b)
whos % empty workspace
% Load
load matlab_intro_ONLYa % this will load only the variable a, with value a = 3
whos
load matlab_intro % this will overwrite the current variable a to restore a = 1, and also load
b = 2
whos
```

2.3 Vector and matrices

```
% Most of the entities in MATLAB are matrices, even real numbers! Indeed:
whos a % is a 1-by-1 matrix

%% Vector and matrix generation
% !Row! vectors can be defined writing the elements separated by a !space!, with square
brackets delimiting the sequence ...
c = [1 2 3]
% ... or, equivalently, using commas instead of the spaces
c = [1, 2, 3]
```

```

% !Column! vectors can be defined writing the elements separated by a !semicolon!, with square
brackets delimiting the sequence
c = [1; 2; 3]

% Transposition is performed with an apostrophe: the following command produces a column
vector
c = [1 2 3]'

% Spaces (or commas) and semicolons can be combined in order to define matrices ...
D = [1 2 3; 4, 5, 6; 7 8 9]
% ... transposition works on matrices too.
D = D'

% When defining a matrix, pay attention to the number of elements in each row/column. They
have to be coherent.
D_wrong = [1 2; 3 4 5]

% There are other ways to create vectors and matrices: if you are interested in vectors with
evenly spaced elements it is possible to use the following syntax ...
e = [1:1:10] % syntax: [beginning:step_size:end]
f = [1:10:101]
g = [0:0.1:1]
h = [10:-1:1]
e = [1:10] % the step_size can be omitted if step_size = 1. syntax: [beginning:end]
% or the linspace command.
ee = linspace(1,10,10) % sintax: linspace(beginning, end, n_of_elements)
ff = linspace(1,101,11)
gg = linspace(0,1,11)
hh = linspace(10,1,10)

% You can also a zero vector/matrix ...
i = zeros(1,5) % row vector in  $\mathbb{R}^5$ 
j = zeros(5,1) % column vector in  $\mathbb{R}^5$ 
K = zeros(3,4) % matrix in  $\mathbb{R}^{3 \times 4}$ 
L = zeros(2,2) % matrix in  $\mathbb{R}^{2 \times 2}$ 
M = zeros(2) % matrix in  $\mathbb{R}^{2 \times 2}$ 
% .. or ones matrices ..
n = ones(1,4)
O = ones(2)
% .. or identity matrices.
P = eye(3)
Q = eye(3,5) % rectangular too!!
% .. or diagonal matrix
d = [1:4];
D = diag(d)
% Beware of the different of "diag" wheter the argument is a vector (as before) or a matrix
M = [1:3; 4:6; 7:9];
diagM = diag(M) % diagM now contains the diagonal part of the matrix M, ie diagM = [M_ii]_i
Md = diag(diag(M)) % Md is a copy of M, in which a value 0 has been overwritten over the
original value of each element outside the diagonal
% Diag also accepts a second argument. In that case it takes the upper or lower diagonals
Dup = diag(d, 1) % matrix: the values of the first upper diagonal are taken from d
Dbottom = diag(d, -2) % matrix: the values of the second lower diagonal are taken from d
diagMup = diag(M,1) % vector containing the values of the first upper diagonal
diagMbottom = diag(M,-2) % vector containing the values of the second lower diagonal
% tril and triu commands take instead the lower and upper triangular part of a matrix.
Ml = tril(M)

% You can also build vectors and matrices by composing blocks..
r_minus = [-5:1:-1];
r_plus = [0:5];
r = [r_minus, r_plus]
%
S1 = [1 2; 5 6];
S2 = [3 4; 7 8];
S3 = [9, 10, 11];
S4 = [12];
S = [S1, S2; S3, S4]
% ... but when you do this, pay attention to the dimension of each block!
r_wrong = [r_minus, r_plus']

```

```

S_wrong = [S1, S2; S3]

% Clear
clear all

%% Vector and matrix r/w access
% In order to access to vector and matrix elements we use the () operator.
% It works in both directions, access to read and access to write.
% IMPORTANT: MATLAB indexing starts from 1, unlike C++ for example, where indexing starts from
0.
c = [1:3]
c1 = c(1)
%
Q = ey(3,5);
Q11 = Q(1,1)    % Read access
Q12 = Q(1,2)
%
Q
Q(1,1) = 15     % Write access
Q

% You can also access to blocks of the matrices, by indicating the indexes of the rows and
columns you want to consider.
c
c([2,3])
Q
Q([1,2],[1,3,4])

% In this operation the operator : is helpful. You can select ranges of rows and columns ..
Q([1:2], [1:3]) % First two rows and three columns
% .. or leave a mute index, to mean to get all the indices in that direction
Q(:,1:3) % Take all the rows and the first three columns
Q(1,:)   % Take all the columns and just the first row

% The end operator is and indicator for the last index in an indexing
% expression
Q(end, end) % Take the bottom right element of Q.

% Clear
clear all

```

2.4 Vector operations

```

a = [1 -2 1 0];
b = [3 4 1 -2];
c = a + b % vector sum
d = a - b % vector difference

% Now we would like to perform the scalar product between vectors a and b.
% Try
e = a*b
% MATLAB says that the dimensions of a and b have something wrong, even if they both have 4
elements, as can be seen with
length(a)
numel(b) % length and numel are equivalent for vectors
% Remember that MATLAB stores vectors as matrices and for this reason they have to satisfy the
requirements for matrix product:
% number of columns of the first matrix = number of rows of the second matrix
% which is clearly violated in this case.

% The scalar product between a and b can be computed with
e = a*b'
% .. or
e = dot(a,b)
% The tensor product  $F_{[ij]} = a_i b_j$  can be computed with
F = a'*b
% Elementwise operations between two vectors are defined !if! the two vectors have the same
number of elements:
% * elementwise product:

```

```

g = a.*b
% * elementwise division:
h = a./b
% * elementwise exponentiation
i = a.^b
j = a.^2

```

2.5 Matrix operations

```

K = [1 -1 2; 4 -2 0];
L = [1 2 2; 3 -3 1];
M = K + L % matrix sum
N = K - L % matrix difference

% If two matrices satisfy the usual dimensional requirements their product can be computed in
MATLAB
O_wrong = K * L % is wrong, as can be seen with the "size" command
size(K)
size(L)
O = K * L' % ok

% Elementwise operations between two matrices are defined !if! the two matrices have the same
size:
% * elementwise product:
P = K .* L
% * elementwise division:
Q = K ./ L
% * elementwise exponentiation
R = K .^ 2
S = K .^ L
% The power matrix operation K^2 = K*K is meaningful only for square matrices.
T = K^2
U = F^2

% Clear
clear all

% The determinant of a matrix is obtained with the det command
A = [1 2 3; -2 3 -1; 0 1 2];
det(A)
% The trace (the sum of the diagonal elements) is obtained with the trace operator.
trace(A)
% In order to compute the inverse of a (small) matrix you can use the inv function.
invA = inv(A)
% Check
A * inv(A) % = identity
% This function is very expensive (and may fail, A*A^-1 not being the identity matrix!) and is
not used in practice: for big matrices and you DO NOT have to use it for solving linear
systems!!
% We will see some numerical methods for the resolution of linear systems. Moreover, the
backslash operator can be used to solve efficiently a linear system
b = [1 2 3]';
x = A \ b % x is the solution of Ax = b
% Check
A * x
b

% Clear
clear all

```

2.6 Plotting

```

% We now describe some essential commands for 2D plotting
% The most simple technique to plot functions is to discretize an interval and then evaluate
the function in the obtained points.
% Plot the function y = x^2 - x + sin(pi*x) in [0,2]
x = 0:0.1:2;

```

```

y = x.^2 - x + sin(pi*x); % NOTICE the use of the .^ operation
figure % Open a figure window
plot(x,y)
% Increase the number of nodes
hold on % plot on the same graph, for a comparison
x2 = 0:0.01:2;
y2 = x2.^2 - x2 + sin(pi*x2);
plot(x2,y2,'r') % 'r' = red color
% Other helpful options:
grid on % draw a grid on the graph
axis([0 2 1 2]) % set the limits on the x and y axes. Syntax: axis([xmin xmax ymin ymax])
axis equal % The scaling of the axes might be different. The same scaling is forced using the
axis equal command

% Clear and close all figures
clear all
close all

```

2.7 Function handles

```

% A function handle is a pointer to another function and it is defined with the syntax
% @function-name

% For example,
f = @sin;
% creates a function handle called f that refers to the function sin (builtin in MATLAB).

% Function handles are used to call other functions indirectly, or to pass a function as an
argument to another function. For example:
x = linspace(0, 2*pi, 1000);
plot(x, f(x), 'Linewidth', 2)

% A function handle can be evaluated writing its name followed by an argument list. If there
are no arguments, you must use an empty argument list '()'. For example:
f(pi/4) %sin(pi/4)

% Anonymous functions are defined using the syntax
% @(argument-list) expression

% Any variable that is not found in the argument list is inherited from the enclosing scope.
Anonymous functions are useful for creating simple unnamed functions from expressions or for
wrapping calls to other functions to adapt them for use by functions. For example:
f = @(x) x.^2;
hold on
plot(x, f(x), 'r-', 'Linewidth', 2)

% Function handles can be used to reshape the form of a function in order to satisfy the
requirements of another function. As an example, quad is a function for performing numerical
integration. It needs single variable functions. Suppose we have
g = @(x,y) sin(x) .* cos(y);
% and we need to compute the integral for x in [0, 1] and y = 0.
% With the function handles we can transform a two-argument function f into a single-argument
one with
y = 0;
quad(@(x) g(x, y), 0, pi)
% Clear and close all figures
clear all
close all

```

2.8 Logicals

```

% Sometimes you have to work with logical conditions (expression that return either true or
false).
% True is associated with 1 and false with 0
true
false

```

```

% MATLAB offers some logical operators (e.g. equal, less, and, or, ...)
a = 1;
b = 2;
c = 3;
d = 2;
% Logical operators
a < b % Smaller than
b > c % Greater than
d <= b % Smaller or equal than
d == b % Equal to
a ~= b % Different from % IMPORTANT: tilde with alt+0126 on Windows
(a < b) & (b > c) % And
(a < b) | (b > c) % Or

```

2.9 Control flow statements

2.9.1 if-then-else statements

```

% The if instruction allows to execute a sequence of commands if a particular logical
condition is satisfied.
% Syntax:
% if (condition1)
%   block1
% elseif (condition2)
%   block2
% else
%   block3
% end

% Compute the factorial of a number n
n = 5;
%n = -5;
if (n < 0)
    disp('Negative n. Undefined factorial.')
elseif (n == 0)
    factorial = 1
else
    factorial = prod(1:n)
end

```

2.9.2 for loops

```

% The for loop iterates a series of operations for a fixed number of times
% Syntax:
% for (index = start:step:end)
%   instruction block
% end

% Sum the first 10 integer numbers
a = 0;
for (i = 1:10)
    a = a + i;
end
a

% Nested loops: compute the Hilbert matrix of order n=4
n = 4;
A = zeros(n);
for i = 1:n
    for j = 1:n
        A(i,j) = 1 / (i + j - 1);
    end
end
A

```

2.9.3 while loops

```
% The while loop repeats a sequence of operations until a logical condition becomes false.
% Syntax:
% while (condition)
%   instruction block
% end

% Sum the first 10 integer numbers
a = 0;
i = 1;
while (i < 11)
    a = a + i;
    i = i + 1;
end
a
i % should be 11

% ATTENTION! If for any reason (most of the times a programming error) the logical condition
is ALWAYS satisfied, the loop is infinite. In order to stop the iterations you have to press
ctrl+c!
while (true)
end
```

2.10 M-files

```
% There are two categories of m-files.
% SCRIPTS. These files contain instructions that are executed in sequence in the command line
if the script file is called.
edit matlab_intro_script.m
% The content of matlab_intro_script.m is:
% =====
% a = 1;
% b = 2;
% c = a + b;
% =====
matlab_intro_script
% The variables a, b and c are saved in the current workspace
a
b
c

% FUNCTIONS. Functions take some input arguments and return some outputs after a series of
instructions are performed.
% Notice the particular syntax with the FUNCTION and END keywords.
edit matlab_intro_function.m
% The content of matlab_intro_function.m is:
% =====
% function sum = matlab_intro_function(num1, num2)
%   sum = num1 + num2;
% end
% =====
% The variables defined in the function are LOCAL to the SCOPE of the function itself. There
is no variable "sum"; "c" contains the output of the function
c = matlab_intro_function(a, b)
whos sum
```

3 Basic MATLAB usage

3.1 Basic commands

```
>> ver
-----
MATLAB Version: 23.2.0.2485118 (R2023b) Update 6
MATLAB License Number: 924786
```

`ver` shows the Matlab version we are working on

```
>> help sqrt
sqrt    Square root.
      sqrt(X) is the square root of the elements of X. Complex
      results are produced if X is not positive.

      See also sqrtm, realsqrt, hypot.
```

help shows how to use a function

```
>> doc sqrt
>>
```

and doc displays online documentation about a command

```
octave:1> pi
ans = 3.1416
octave:2> i
ans = 0 + 1i
octave:3> inf
ans = Inf
```

Some constants are allocated by default

Default Mathematical Functions

Matlab provides a large number of standard elementary mathematical functions. To have more information, type the command: `help elfun`

It is convenient to change the default format used by MATLAB to represent the number. Variables visualization can be changed by using the command `format` as following:

- **format short** fixed comma with 5 digits(default): `pi=3.1416`
- **format long** fixed comma with 15 digits(default): `pi=3.14159265358979`
- **format short e** mobile comma with 5 digits(default): `pi=3.1416e+00`
- **format long e** mobile comma with 15 digits(default): `pi=3.141592653589793e+00`

3.2 First MATLAB/Octave - Exercises

Exercise 1.1. Define the row vector

$$\underline{v}_k = [1, 9, 25, \dots, (2k+1)^2] \in \mathbb{R}^{k+1}$$

with $k = 8$ using the following strategies:

1. A `for` loop to define one by one each element of the vector.
2. The vector syntax to build it in just one shot.

Solution Exercise 1.1.

```

%% Exercise 1.1
%% Define the Row Vector vk=(1,9,25...(2k+1)^2) with k=8
k = 8;

% 1) For loop strategy
vk = zeros(1, k+1);
fprintf("Loop strategy: \n");
tic
for (ii = 0:k)
    vk(ii+1) = (2*ii + 1)^2;
end
toc
% 2) Vector syntax
fprintf("Vector syntax: \n");
tic
vk = [1:2:2*k+1].^2;
toc

%% Suggestion: Test the difference in computational time for increasing k

```

Exercise 1.2. Define a function which, for an input value k , returns the corresponding vector $\underline{v}_k$ as defined in the previous exercise.

Solution Exercise 1.2.

```

%% Exercise 1.2
% Define a function which, for an input value k, returns the corresponding
% vector v k as defined in the previous exercise.

function vk = ex_1_2(k)

    vk = [1:2:2*k+1].^2;

end

```

Exercise 1.3. Using the function of the previous exercise write another function that returns, for a generic value k , the $2(k+1) \times 2(k+1)$ matrix

$$M_k = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & \sqrt{2} & 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & \sqrt[3]{2} & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & \sqrt[4]{2} & 0 & 0 & \dots & 0 & 9 \\ 0 & 0 & 0 & 0 & \sqrt[5]{2} & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \sqrt[6]{2} & \dots & 0 & 25 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \dots & \sqrt[2k+1]{2} & 0 \\ 1 & 1 & 9 & 9 & 25 & 25 & \dots & (2k+1)^2 & (2k+1)^2 \end{bmatrix}.$$

Solution Exercise 1.3.

```

function Mk = ex_1_3(k)

    % Create a diagonal matrix with the right elements and dimension
    Mk = diag(2.^(1./[1:2*(k+1)]));
    % The bottom right element will be overwritten

    % Last column
    Mk(2:2:end, end) = ex_1_2(k);

    % Last line
    Mk(end, 1:2:end) = ex_1_2(k);
    Mk(end, 2:2:end) = ex_1_2(k);

end

```

4 Floating-point numbers

4.1 Representation of floating-point numbers

Exercise 1.4. Compare the results of the following code segments:

Code A

```
x = 0;
while (x ~= 1)
    x = x + 1/16
end
```

Code B

```
x = 0;
while (x ~= 1)
    x = x + 0.1
end
```

Solution Exercise 1.4. Code A works as expected: the **while** loop is repeated 16 times, and the final value of x is 1.

Instead, code B does **not** work as expected, and results in an infinite loop.

Computing environments use floating-point arithmetic, which involves a **finite set of numbers** with **finite precision**.

Given a real number x , the typical floating-point implementation represents its identifier $fl(x)$ in the following **normalized** form:

$$fl(x) = (-1)^s \cdot (0.a_1a_2 \dots a_t) \cdot b^e$$

where

- s is the **sign**;
- b is the **base** of the system;
- e is the **exponent**, $e \in [L, U]$, $L < 0$, $U > 0$;
- $m = a_1a_2 \dots a_t$, with $0 \leq a_i \leq b - 1$, is the **mantissa** or **fractional part**. To have a **unique representation** for each number we force $a_1 \neq 0$.¹

MATLAB represent floating point numbers in **double-precision** format:

- binary representation: $b = 2$;
- 64-bit words, with
 - 1 bit for the sign: 0 (+) or 1 (–);
 - $t = 52$ bits for the mantissa: $0 \leq m < 2^{52} \rightarrow$ limitation on the **precision** of the arithmetic;
 - 11 bits for the exponent: $L = -1022, U = 1023$ ² $\rightarrow$ limitation on the **range** of the arithmetic;
- the minimum and the maximum normalized machine numbers (in absolute value) that can be represented in double precision can be obtained in MATLAB with

$$\begin{array}{ll} \text{realmin} & \% = 2^{L-1} = 2.2251 \cdot 10^{-308} \\ \text{realmax} & \% = (1 - 2^{-t}) \cdot 2^U = 1.7977 \cdot 10^{308}. \end{array}$$

¹Otherwise, e.g. in decimal basis, the number $1/10$ could be represented as $0.1 \cdot 10^0$, but also as $0.01 \cdot 10^1$, but also as ...; for this reason, the representation is called **normalized**. As a consequence, a special coding has to be devised to represent the number 0

²What is actually stored is not the exponent e but the biased exponent $E = e + \text{bias}$, with $\text{bias} = 2^{11-1} - 1 = 1023$. This accommodates for a positive sign of the biased exponent, avoiding the need of an additional bit for the storage of the sign. Extremal values of E are reserved to encode ± 0 , $\pm \infty$, NaN.

- Any computation trying to produce a value larger than **realmax** is said to **overflow**. The result is set to **Inf**.
- Any computation trying to produce a value smaller than the minimum number is said to **underflow**. If possible the result is treated in a special way (denormalized number), otherwise is set to 0.
- For any computation trying to produce a value that is undefined the result is called NaN. (e.g. 0/0, Inf/Inf, Inf-Inf).

The different behaviour of Codes A and B can be now explained: the binary representation of $1/16$ is

$$\frac{1}{16} = 2^{-4}$$

and is therefore stored exactly in a double precision representation. Instead, the binary representation of $1/10$ is

$$\frac{1}{10} = 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots$$

and **cannot be stored exactly** in any finite-precision representation.

4.2 Floating-point arithmetic

Definition 1.1 (Machine epsilon). *The smallest positive machine number that, if added to 1, gives a result greater than 1. $\text{eps} = 2^{-t}$ Note that **eps** is different from **realmin**. In MATLAB you can get the machine number by using the **eps** command.*

Exercise 1.5 (Machine epsilon). Find the machine epsilon by implementing an **ad hoc procedure**. Comment and justify the obtained results.

Solution Exercise 1.5.

```
%% Exercise 1.5
% Pay attention, you cannot use "eps" since it is a built-in variable
k = 0;
EPS = 1/2;
while (1 + EPS) > 1
    EPS_old = EPS; % keep track of the value
    EPS = EPS / 2;
    k = k + 1;
end
EPS_old % computed with the ad-hoc procedure
k % Number of iterations, which is also the number of digits in the mantissa, according
to the standard
eps % MATLAB built-in
```

Exercise 1.6 (Overflow and associative property). **realmax**

```
a = 1.0e+308;
b = -a;
c = 1.1e+308;
(a + b) + c
(a + c) + b
```

Solution Exercise 1.6. While commutative property still holds ($a + b = b + a$ and $a \cdot b = b \cdot a$), associative property is no longer valid within floating-point arithmetic ($(a + b) + c \neq a + (b + c)$). The **order** in which the operations are performed sometimes matters, especially when overflow or underflow exception may occur!

Exercise 1.7 (Numerical cancellation). Consider the following function

$$f(x) = \frac{e^x - 1}{x}$$

- Evaluate $f(x)$ for values of x around zero (try with $x_k = 10^{-k}$, $k \in [1, 20]$). What do you obtain? Explain the results.
- Propose an approach for fixing the problem. (Hint: Use Taylor expansions to get an approximation of $f(x)$ around $x = 0$.)
- How many terms in the Taylor expansion are needed to get double precision accuracy (16 decimal digits) $\forall x \in [0, 1/2]$?

Solution Exercise 1.7.

- Evaluate $f(x)$ for values of x around zero (try with $x_k = 10^{-k}$, $k \in [1, 20]$). What do you obtain? Explain the results.

```
k = [1:20]';
x = 10.^(-k);
f = @ (x) (exp(x) - 1) ./ x;
format long
[k x f(x)]
```

The computed $f(x_k)$ have the expected behavior until $k = 7$, in agreement with

$$\lim_{x \rightarrow 0} f(x) = 1.$$

Then, from $k = 8$ to $k = 15$, the sequence oscillates with increasing amplitude. Then, at $k = 16$, the result becomes 0! The cause of these effects is the **loss of accuracy** due to numerical cancellation. For $k = 8, \dots, 15$ e^{x_k} and 1 are very close to each other, still being different in floating point arithmetic. We are subtracting them and the result suffers of a great relative error. Moreover, we are dividing by x_k , that approaches 0, hence the error is magnified. Finally, for $k = 16$, e^{x_k} and 1 are too close to have a different representation in the floating point arithmetic ($\text{eps} \approx 10^{-16}$), hence the difference is 0.

- Propose an approach for fixing the problem. (Hint: Use Taylor expansions to get an approximation of $f(x)$ around $x = 0$.)

Write the Taylor expansion of e^x around $x_0 = 0$, truncating it at the fifth order:

$$\begin{aligned} e^x &= \sum_{k=0}^{+\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k = \sum_{k=0}^5 \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + O(x^6) = \\ &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + O(x^6) \end{aligned}$$

and substitute it in $f(x)$

$$f(x) = 1 + \frac{x}{2} + \frac{x^2}{6} + \frac{x^3}{24} + \frac{x^4}{120} + O(x^5)$$

Within this formula, at least for $x > 0$, all the terms have the same sign, so no numerical cancellation occurs.

```
f_taylor_5 = @ (x) 1 + 1/2*x + 1/6*x.^2 + 1/24*x.^3 + 1/120*x.^4;
[k x f(x) f_taylor_5(x)]
```

- How many terms in the Taylor expansion are needed to get double precision accuracy (16 decimal digits) $\forall x \in [0, 1/2]$?

The error in the approximation of a function $g(x)$ at a point $x = a$ with its Taylor expansion up to order n is

$$E[g, n, a](x) = \frac{g^{(n+1)}(\xi)}{(n+1)!} (x-a)^{n+1} \quad \text{for at least a } \xi \text{ s.t. } |\xi - a| \leq |x - a|$$

In this case

$$E[e^x, n+1, 0](x) = \frac{e^\xi}{(n+2)!} x^{n+2} \quad \text{for } \xi \in [0, x]$$

and therefore, substituting in $f(x)$

$$E[f, n, 0](x) = \frac{E[e^x, n+1, 0](x)}{x} = \frac{e^\xi}{(n+2)!} x^{n+1} \quad \text{for } \xi \in [0, x]$$

We want

$$E[f, n, 0](x) < \text{eps} \quad \forall x \in \left[0, \frac{1}{2}\right] :$$

since both e^ξ and x^{n+1} are increasing functions, it's enough to choose $\xi = x = \frac{1}{2}$ and seek the first $\tilde{n}$ such that

$$\frac{e^{1/2}}{(n+2)!} (1/2)^{n+1} < \text{eps}$$

```
format short e
n = [1:20]';
err = 1./factorial(n+2) .* (0.5).^(n+1) .* exp(0.5);

[n err]
```

Therefore $\tilde{n} = 13$.

Exercise 1.8 (Error propagation). The sequence

$$1, \frac{1}{3}, \frac{1}{9}, \dots, \frac{1}{3^n}, \dots$$

can be generated with the following recursive relations

$$\begin{cases} p_n = \frac{10}{3} p_{n-1} - p_{n-2} \\ p_1 = \frac{1}{3}, p_0 = 1 \end{cases} \quad \begin{cases} q_n = \frac{1}{3} q_{n-1} \\ q_0 = 1. \end{cases}$$

- Implement the two relations in order to generate the first 100 terms of the sequence.
- Study the stability of the two algorithms and justify the obtained results.

Solution Exercise 1.8.

- Implement the two relations in order to generate the first 100 terms of the sequence.

```
% First recursive relation
p(1) = 1;
p(2) = 1/3;
for i = 2:100
    p(i+1) = 10/3*p(i) - p(i-1);
end

figure
subplot(2,1,1)
plot(0:100, p, 'LineWidth', 3)
set(gca, 'FontSize', 16)
```

```

xlabel('n','FontSize',16)
ylabel('p_n','FontSize',16)
% The sequence explodes!

% Second recursive relation
q(1) = 1;
for i=1:100
    q(i+1) = 1/3*q(i);
end

subplot(2,1,2)
plot(0:100, q, 'LineWidth',3)
set(gca,'FontSize',16)
xlabel('n','FontSize',16)
ylabel('q_n','FontSize',16)

% The sequence instead is ok.

```

b. Study the stability of the two algorithms and justify the obtained results.

Error analysis: since we use finite precision numbers, we know the initial data up to an error ε ³Indeed, denoting by $fl(p_i)$ the floating point representative for p_i

$$fl(p_0) = p_0 + \varepsilon, \quad fl(p_1) = p_1 + \varepsilon.$$

It follows

$$\begin{aligned}
 fl(p_2) &= \frac{10}{3} fl(p_1) - fl(p_0) = \frac{10}{3}(p_1 + \varepsilon) - (p_0 + \varepsilon) = p_2 + \frac{7}{3}\varepsilon \\
 fl(p_3) &= \frac{10}{3} fl(p_2) - fl(p_1) = \frac{10}{3}(p_2 + \frac{7}{3}\varepsilon) - (p_1 + \varepsilon) = p_2 + \frac{61}{9}\varepsilon \\
 \tilde{p}_4 &= \dots
 \end{aligned}$$

The error is amplified! Instead, with the second formula

$$\begin{aligned}
 fl(q_0) &= q_0 + \varepsilon \\
 fl(q_1) &= \frac{1}{3} fl(q_0) = \frac{1}{3} q_0 + \frac{1}{3} \varepsilon = q_1 + \frac{1}{3} \varepsilon \\
 fl(q_2) &= \frac{1}{3} fl(q_1) = \frac{1}{3} q_1 + \frac{1}{9} \varepsilon = q_2 + \frac{1}{9} \varepsilon \\
 fl(q_3) &= \dots
 \end{aligned}$$

The error is reduced!

³Actually, if we start from 1, we know it exactly, but we may introduce anyway errors in the following steps.